

Instituto Tecnológico y de Estudios Superiores de Monterrey

Código de experimentación base con MLFlow

Iván Plata Garnica - A01840199

TC5061 - Operaciones de aprendizaje automático
09 de Octubre de 2026

Nota. Repositorio individual de GitHub

El código fuente de la actividad, incluido el script de entrenamiento y los archivos relacionados con el proyecto, se encuentra disponible en el siguiente repositorio:

https://github.com/iplatamx/TC5061-semana-04/

1. Análisis de resultados

1.1. Mejor configuración

La comparación de las cinco configuraciones experimentales registradas en MLflow muestra que la configuración con $C = 0{,}25$, solver `lbfgs`, `max_iter`=1000 y `random_state`=7 obtuvo la mayor exactitud (*accuracy* = 0.809) y el mejor ROC-AUC (0.857).

Por otra parte, la configuración con $C = 1{,}50$, solver `newton-cg`, `max_iter`=1500 y `random_state`=42 obtuvo el mejor F1-score (0.609). Por lo tanto, la elección de la configuración depende de la métrica utilizada para evaluar el modelo.

1.2. Influencia de los hiperparámetros

No es posible determinar de manera concluyente cuál hiperparámetro tuvo mayor influencia en el desempeño, debido a que las configuraciones evaluadas modificaron simultáneamente varios parámetros, entre ellos C, el algoritmo de optimización y el número máximo de iteraciones.

Para aislar la influencia de cada hiperparámetro, sería necesario realizar experimentos controlados, manteniendo constantes los demás parámetros y modificando únicamente uno por vez.

1.3. Limitaciones observadas

Aunque las configuraciones obtuvieron valores de ROC-AUC superiores a 0.84, el recall se mantuvo alrededor de 0.56. Esto indica que el modelo identifica aproximadamente el 56 % de los clientes que realmente abandonan el servicio, por lo que una proporción importante de estos casos no es detectada.

Esta limitación es relevante para un problema de predicción de abandono, ya que identificar a los clientes en riesgo permite orientar acciones de retención.

Además, los experimentos consideran cinco configuraciones y una partición de entrenamiento y prueba. Para evaluar la estabilidad de los resultados, sería conveniente ampliar la búsqueda de hiperparámetros y aplicar validación cruzada.

1.4. Evidencias de los experimentos en MLflow

Para respaldar el análisis de resultados, se presentan dos capturas obtenidas de MLflow. La primera muestra la comparación de las configuraciones experimentales; la segunda documenta el detalle de una corrida, incluidos sus parámetros, métricas y artefactos.

Evidencia 1. Tabla comparativa de corridas

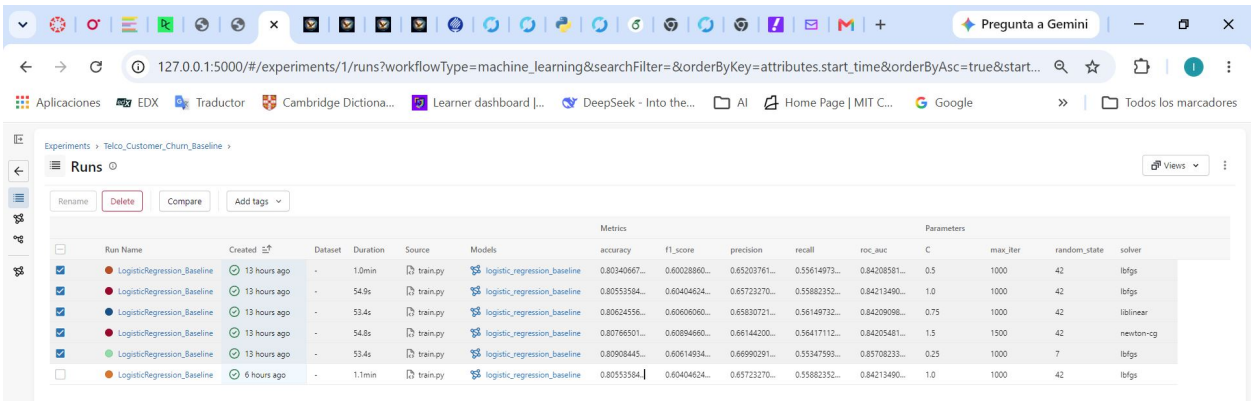


Figura 1: Tabla comparativa de las configuraciones experimentales registradas en MLflow.

Evidencia 2. Detalle de una corrida y sus artefactos

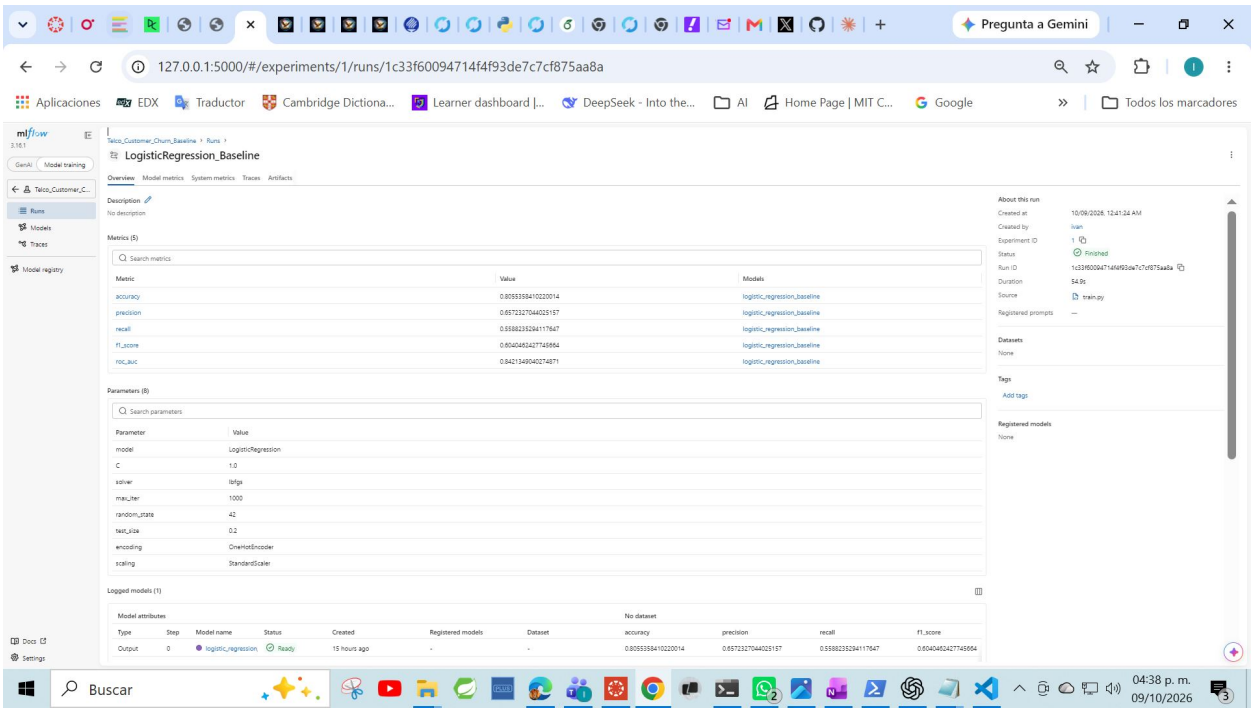


Figura 2: Detalle de una corrida experimental y sus artefactos registrados en MLflow.

2. Documentación de la semilla y las dependencias

2.1. Fijación de la semilla aleatoria

Para favorecer la reproducibilidad de los experimentos, se utilizó el parámetro `random_state`, cuyo valor se establece mediante argumentos de línea de comandos. La semilla se utiliza tanto en la partición de entrenamiento y prueba como en la configuración del modelo de regresión logística.

El siguiente comando ejecuta el entrenamiento con una semilla fija de 42:

```
python train.py --c 1.0 --solver lbfgs --max-iter 1000 --random-state 42
```

2.2. Validación de la reproducibilidad

Se ejecutó dos veces el script con los mismos argumentos para verificar la consistencia de los resultados. En ambas ejecuciones se obtuvieron las siguientes métricas:

- Accuracy: 0.8055
- Precision: 0.6572
- Recall: 0.5588
- F1-score: 0.6040
- ROC-AUC: 0.8421

Los resultados coincidentes muestran que, bajo las mismas condiciones de ejecución y con los mismos argumentos, el entrenamiento produjo las mismas métricas de evaluación. Esta prueba respalda la reproducibilidad del experimento en el entorno utilizado.

2.3. Documentación de dependencias

Las dependencias del proyecto se registraron en el archivo `requirements.txt`, generado a partir del entorno virtual de Python mediante el siguiente comando:

```
python -m pip freeze > requirements.txt
```

Entre las dependencias principales del proyecto se encuentran:

- Python 3.12.3
- pandas 3.0.6
- scikit-learn 1.9.1
- MLflow 3.16.1

El archivo `requirements.txt` registra los paquetes instalados y sus versiones. Para instalar las dependencias en un entorno virtual, se utiliza el siguiente comando:

```
python -m pip install -r requirements.txt
```

El archivo `requirements.txt` se incluye en el repositorio junto con el script `train.py`, para documentar las dependencias y facilitar la reconstrucción del entorno utilizado.

Referencias

1. MLflow. (s. f.). *MLflow Tracking*. https://mlflow.org/docs/latest/tracking/